

A

Amigo

Smart Local Service Orchestrator

Development, API, usage, testing, and deployment guide

Project Documentation • 2026

Contents

- Features
- Repository structure
- Technology stack
- Prerequisites
- Backend installation
- Database
- Start the backend
- Authentication and security
- API reference
- Run the web frontend
- Optional mobile frontend
- Automated tests and quality checks
- Manual end-to-end checklist
- Troubleshooting
- Security and Git rules
- Current limitations and future improvements
- Documentation commit

Amigo is an AI-assisted local service discovery and booking application. A user describes a need in English, Urdu, or Roman Urdu. The backend extracts intent, finds matching providers from SQLite, ranks them by rating, creates bookings, and records a trace of the orchestration steps.

Features

- Registration and login with JWT bearer authentication.
- PBKDF2-HMAC-SHA256 password hashing with random salts.
- User-owned booking access controls.
- Gemini structured intent extraction.
- Provider filtering and rating-based ranking.
- Booking creation, status updates, and cancellation.
- Traceable orchestration logs.
- Responsive Amigo web frontend and optional Expo mobile client.
- English, Urdu, and Roman Urdu request support.

Repository structure

```
local-service-orchestrator/  
├── backend/           FastAPI application, routes, schemas, and tests  
├── database/         SQLite schema and seed data  
├── frontend/         Current React/TypeScript web application  
├── mobile/           Optional Expo/React Native client  
├── docs/             Project documentation  
└── .env              Local secrets; never commit
```

Technology stack

Backend: Python, FastAPI, Uvicorn, SQLite, Pydantic, JWT, PBKDF2-HMAC-SHA256, Gemini API, Pytest, and HTTPX.

Frontend: React, TypeScript, TanStack Start/Router, Vite, Tailwind CSS, Lucide icons, ESLint, and Prettier.

Prerequisites

Install Python 3.11+, Node.js/npm, Git, and obtain a Gemini API key for live AI extraction. Automated tests mock Gemini and do not require network access.

Backend installation

```
cd ~/local-service-orchestrator  
python3 -m venv .venv  
source .venv/bin/activate  
pip install -r backend/requirements.txt  
cp backend/.env.example .env
```

Configure .env without committing it:

```
SECRET_KEY=replace-with-a-long-random-secret
```

```
DATABASE_PATH=database/khidmat.db
GEMINI_API_KEY=your-gemini-key
GEMINI_MODEL=gemini-3.6-flash
CORS_ORIGINS=http://localhost:5173,http://localhost:8081,http://localhost:19086
```

Never include a real API key in documentation, source code, screenshots, or Git commits.

Database

The SQLite schema contains:

- users: user ID, name, email, and password hash.
- providers: name, category, neighborhood zone, and rating.
- bookings: user/provider relationship, booking time, and status.
- orchestration_requests: original request and workflow status.
- orchestration_logs: workflow events and JSON details.

Supported categories:

```
plumber, electrician, tutor, carpenter, painter
```

Supported areas:

```
Gulshan, Johar, Clifton, DHA, Nazimabad
```

Booking statuses:

```
pending, confirmed, completed, cancelled
```

Create a fresh local database manually when required:

```
cd ~/local-service-orchestrator
sqlite3 database/khidmat.db < database/schema.sql
sqlite3 database/khidmat.db < database/seed_data.sql
```

Seed passwords are mock development data only and must not be used in production.

Start the backend

Use port 8002 for this project:

```
cd ~/local-service-orchestrator
source .venv/bin/activate
python -m uvicorn backend.main:app \
  --reload \
  --env-file .env \
  --host 0.0.0.0 \
  --port 8002
```

Health check:

```
curl http://127.0.0.1:8002/health
```

Expected response:

```
{"status":"ok"}
```

Interactive API documentation is available at:

```
http://127.0.0.1:8002/docs
```

Authentication and security

Registration hashes passwords using PBKDF2-HMAC-SHA256 with a random salt. Only the encoded hash is stored. Login verifies the password and returns a JWT.

Protected requests use:

```
Authorization: Bearer ACCESS_TOKEN
```

The backend obtains `user_id` from the validated JWT and applies it to every protected booking/request query. Clients cannot submit another user's ID to access their data.

API reference

Base URL: `http://127.0.0.1:8002`

GET /health

Public health check. Returns `{"status":"ok"}`.

POST /register

Request:

```
{
  "name": "Terminal Tester",
  "email": "tester@example.com",
  "password": "secure-password"
}
```

Returns 201 Created and the new user's ID, name, and email. The password is never returned.

POST /login

Request:

```
{
  "email": "tester@example.com",
  "password": "secure-password"
}
```

Returns:

```
{
  "access_token": "JWT_TOKEN",
  "token_type": "bearer"
}
```

GET /bookings

Authenticated endpoint. Returns only the current user's bookings:

```
[
  {
    "booking_id": 1,
    "user_id": 1,
    "provider_id": 2,
    "provider_name": "Fast Fix Electricians",
    "category": "electrician",
    "neighborhood_zone": "Johar",
    "booking_time": "2026-08-10T14:30:00",
    "status": "pending"
  }
]
```

POST /bookings

Authenticated endpoint. Creates a booking:

```
{
  "provider_id": 2,
  "booking_time": "2026-08-10T14:30:00"
}
```

Returns 201 Created with the created booking.

PATCH /bookings/{booking_id}

Authenticated ownership-protected endpoint. Updates a status:

```
{"status": "confirmed"}
```

Allowed values are pending, confirmed, completed, and cancelled.

DELETE /bookings/{booking_id}

Authenticated ownership-protected endpoint. Marks the booking as cancelled and retains it for history.

POST /service-requests

Authenticated natural-language orchestration endpoint:

```
{
  "message": "I need an electrician in Johar on 2026-08-10 at 14:30"
}
```

The response includes request_id, status, extracted intent, ranked providers, and the workflow trace.

Example intent:

```
{
  "category": "electrician",
  "neighborhood_zone": "Johar",
  "requested_date": "2026-08-10",
  "requested_time": "14:30",
  "language": "english"
}
```

Possible request statuses are ready, needs_clarification, failed, and booked. Missing service type or area results in needs_clarification. Missing Gemini configuration returns 503; invalid extraction returns 502.

GET /service-requests/{request_id}/trace

Authenticated ownership-protected endpoint. Returns the original input, extracted intent, matching providers, and trace events such as:

```
request_received
intent_extracted
providers_filtered
providers_ranked
booking_created
```

POST /service-requests/{request_id}/book

Authenticated ownership-protected endpoint. The selected provider must match the request's extracted category and neighborhood.

```
{
  "provider_id": 2,
  "booking_time": "2026-08-10T14:30:00"
}
```

booking_time can be omitted when the request contains an exact date and time. New bookings start as pending, and the orchestration request becomes booked.

Run the web frontend

```
cd ~/local-service-orchestrator/frontend
npm install
cp -n .env.example .env
```

Set frontend/.env:

```
VITE_API_URL=http://localhost:8002
```

Start the frontend:

```
npm run dev
```

Open the Vite URL, normally <http://localhost:5173>.

User flow:

```
Welcome → Register/Login → Ask for help → Provider results → Booking → My Bookings
```

The web client stores the JWT in browser local storage and attaches it to protected requests. A 401 response clears the session and sends the user back to login.

Optional mobile frontend

```
cd ~/local-service-orchestrator/mobile
npm install
cp -n .env.example .env
npx expo start
```

For a physical phone, use the computer LAN IP instead of localhost:

```
EXP0_PUBLIC_API_URL=http://192.168.0.115:8002
```

The phone and computer must share a network. Run Uvicorn with `--host 0.0.0.0`.

Automated tests and quality checks

Backend tests use a temporary SQLite database and an internal HTTPX ASGI client. Gemini is mocked where appropriate, so tests do not consume API quota.

Run backend tests:

```
cd ~/local-service-orchestrator
source .venv/bin/activate
PYTHONPATH=. pytest -q backend/tests
```

The test suite covers registration, login, password protection, duplicate accounts, authentication requirements, booking creation, ownership checks, intent extraction, provider ranking, trace logs, missing AI configuration, and clarification handling.

Run Python and repository checks:

```
python3 -m compileall -q backend
git diff --check
```

Run frontend checks:

```
cd ~/local-service-orchestrator/frontend
npm run build
npm run lint
```

npm run build verifies the production client/server bundle. npm run lint checks frontend code quality and formatting. A React Fast Refresh warning in src/lib/auth.tsx is non-blocking when there are zero lint errors.

Manual end-to-end checklist

- 1 Start the backend and frontend.
- 2 Register a new user with a password of at least eight characters.
- 3 Log in and submit an English request.
- 4 Confirm providers appear in rating order.
- 5 Choose a provider and create a booking.
- 6 Open ****My Bookings**** and refresh the list.
- 7 Submit a Roman Urdu or Urdu request.
- 8 Submit an incomplete request and verify the clarification message.
- 9 Try an incorrect password and verify the friendly error.
- 10 Log out and verify protected pages require login.

Troubleshooting

Port 8002 is busy

```
ss -ltnp | grep :8002
```

Use another backend port and update frontend/.env if necessary.

Frontend cannot reach the backend

Check that the backend is running, /health succeeds, VITE_API_URL uses the correct port, CORS includes the frontend origin, and a physical phone uses the computer's LAN IP.

Gemini extraction fails

```
grep -q '^GEMINI_API_KEY=.' .env && echo "Gemini key is configured"
grep '^GEMINI_MODEL=' .env
```

Restart Uvicorn after changing .env. Never print the key itself.

Registration returns 422

Check the name, email format, and password length. The backend requires at least eight password characters.

Registration returns 409

The email already exists. Use a different test email.

Frontend lint reports formatting errors

```
cd ~/local-service-orchestrator/frontend
npm run lint -- --fix
npm run lint
```


Review automatic changes before committing.

Security and Git rules

- Never commit .env files or API keys.
- Never store plaintext passwords.
- Use a strong SECRET_KEY outside local development.
- Preserve ownership filters on protected queries.
- Validate external data with Pydantic.
- Do not use seed credentials in production.
- Do not commit node_modules, Python caches, SQLite databases, Expo output, or frontend build output.

Current limitations and future improvements

The current project uses mock provider data and SQLite. Future improvements can include real provider onboarding, availability, notifications, reminders, refresh tokens, PostgreSQL, rate limiting, automated browser tests, reviews, provider profiles, and administration tools.

Documentation commit

After reviewing this document and running the checks:

```
cd ~/local-service-orchestrator
git add docs/PROJECT_DOCUMENTATION.md
git commit -m "Add complete Amigo project documentation"
```

Do not add .env, database files, API keys, node_modules, or generated build directories.